DeepSeek-V4 Architecture | 12.

Chapter Abstract. DeepSeek-V4 changes the checkpoint's forward path through hybrid CSA/HCA attention and mHC residual mixing. The attention path compresses long sequences while retaining an exact local window; the residual path expands one carrier into four lanes and constrains how sublayers read, mix, and return them. DeepSeekMoE and multi-token prediction remain inherited components rather than additional V4 architecture inventions [76]. This chapter follows one token through those operators in V4-Pro and V4-Flash. Training, agent behavior, cache accounting, and speculative decoding receive their own chapters so optimizer, policy, and serving state are not mistaken for checkpoint sublayers.

One architectural contract ties the operators together: attention may gather only causal selected positions, and the composed residual and expert paths must return a state of the declared width. Section 12.7 reassembles this token path after each operator is inspected separately.

The training process that produces the checkpoint weights is developed in Chapter 13. The model-to-application agent path belongs to Chapter 14. Cache accounting and serving belong to the V4 runtime chapter.

12.1. V4-Pro and V4-Flash in the Family

DeepSeek-V4 extends the V3/V3.2 line under four coupled pressures. Long context must not inherit dense attention cost. Sparse capacity must not become dense per-token work. Deeper learned pathways must remain stable, and specialized post-training must still produce one deployable policy.

The running example is a source-backed V4-Pro/Flash mechanism ledger with two linked lanes. Its architecture lane follows one token through a V4 block: attention must expose the selected context state, mHC must carry and remix four residual lanes, the feed-forward path must preserve route identity, and the block must return a residual state of the declared width. Its training lane follows one rectangular matrix through a Muon update, then records where QAT and specialist consolidation change the state or its owner. The displayed matrices, gates, and route sets are scaled down for calculation; the released Pro/Flash configurations and the report's mechanism order remain the anchor. Training must make the same path stable without turning every sparse or low-precision operation into dense, higher-precision work.

The official report identifies three headline V4 upgrades: hybrid attention with Compressed Sparse Attention (CSA) and Heavily Compressed Attention (HCA), Manifold-Constrained Hyper-Connections (mHC), and the Muon optimizer. That three-part list is an upgrade hierarchy, not the entire model or serving stack. V4 retains the Transformer, DeepSeekMoE, and Multi-Token Prediction (MTP) lineage; its report says the MTP configuration is unchanged from V3. The full sequential MTP mechanism and target alignment are therefore defined in Definition 5.5.1 rather than repeated here [76].

The hybrid attention path interleaves CSA and HCA. CSA compresses KV information by token blocks and uses DeepSeek Sparse Attention (DSA) to select a top-k subset of compressed entries; HCA compresses more heavily and attends densely over its shorter compressed sequence. This changes which context state the model retains and reads. DeepSeekMoE separately limits active expert work, while mHC changes the residual carrier.

Muon and FP4 quantization-aware training change the training path. Specialist supervised fine-tuning (SFT) and Group Relative Policy Optimization (GRPO) feed on-policy distillation into a unified model. The Pro and Flash variants expose different scale and selector settings [76, 143].

The preview release describes V4-Pro as a 1.6T-total-parameter model with 49B active parameters per token and V4-Flash as a 284B-total-parameter model with 13B active parameters per token. It also gives the family a one-million-token context window and names token-wise compression plus DSA as a central innovation [19]. These numbers establish scale and capacity; they do not determine workload quality or serving latency.

The open configurations sharpen the comparison. Both variants use `hc_mult=4`, `hc_sinkhorn_iters=20`, six active routed experts per token, and a 1,048,576-position context field. Flash exposes 256 routed experts and `index_topk=512`; Pro exposes 384 routed experts and `index_topk=1024` [75, 73]. Pro and Flash differ in total scale, expert pool, hidden dimensions, depth, and the amount of context selected by the attention indexer.

Table 12.1 locates headline upgrades, inherited model components, training procedures, and optional serving paths against the V3/V3.2 baseline. The mechanisms share one model, but they do not modify the same part of the computation.

Four owning chapters. This architecture chapter follows mHC residual mixing, CSA/HCA attention, and routed expert execution; Section 12.7 reunites those forward operators. The attention curriculum, Muon, low-precision QAT, specialist training, and on-policy consolidation continue in Chapter 13. Model-native tool protocols and application execution continue in Chapter 14. Cache reuse and draft/verify/commit continue in the V4 runtime chapter.

Table 12.2 fixes the symbols reused in the worked examples.

Figure 12.1 depicts the inference composition. Its two principal sublayers remain attention and feed-forward. mHC changes the residual carrier around them rather than becoming a third sublayer after MoE.

The diagram is deliberately checkpoint-facing. It identifies the state a V4 block transforms, not the request policy used to invoke that block. A larger selector or active-parameter ratio constrains the model's operating point but does not predict retrieval, output quality, or latency; those are measured on the runtime path after the artifact and inference policy are fixed.

Baseline component	V4 change	Intended effect	Preserved contract and new obligation
One residual stream around each block	mHC expands and constrains residual-lane mixing	Control signal propagation through depth while retaining several mixing paths	Layer updates still return compatible hidden states; the mixer must satisfy its nonnegative row- and column-sum constraints.
MLA over retained KV entries	CSA/HCA compress sequence state; DSA selects entries in CSA	Reduce long-context cache and attention work	Attention stays causal and must preserve task-relevant evidence; compression fidelity and selector recall become separate checks.
Shared-plus-routed DeepSeek-MoE	Pro and Flash change scale, expert pools, and active routes	Increase stored capacity without applying every expert to every token	The feed-forward output keeps the residual width; route identity, load, dispatch, and combine remain correctness obligations.
V3 sequential MTP module	Retained without a reported configuration change	Keep the auxiliary future-token training path in the released architecture	Depth-to-target alignment, loss ownership, and the boundary between main-model and MTP-module parameters remain explicit.
Higher-precision training and execution paths	Muon and FP4 quantization-aware training alter optimization and numerical representation	Support large-scale training and a lower-precision execution path	Model quality and operator semantics remain acceptance targets; scales, accumulation, layout, and tolerances must be recorded.
Separate domain policies	Specialist SFT and GRPO feed on-policy full-vocabulary distillation	Consolidate mathematics, coding, agent, and instruction-following behavior without averaging checkpoints	Learner contexts, fixed-teacher roles, contribution weights, and evaluation conditions remain distinct parts of the training and evaluation record.

Table 12.1.: V4 changes compared with the V3/V3.2 architecture and execution baseline. Each change addresses a different pressure and introduces a distinct obligation.

Symbol or term	Meaning in the V4 chapter
m, m'	Chapter-local CSA and HCA sequence-compression rates, respectively; the V4 setup uses $m = 4$ and $m' = 128$.
k	In CSA, the number of compressed KV entries retained by the DSA selector for a query.
ρ	Active-parameter ratio $P_{\text{active}}/P_{\text{total}}$.

Table 12.2.: Local notation for the V4 mechanism and workload examples.

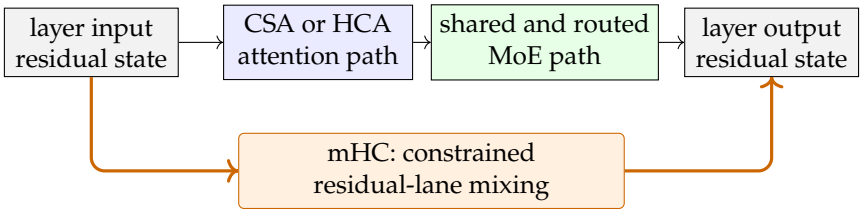


Figure 12.1.: A simplified V4 block boundary. The upper path applies CSA/HCA attention (with DSA selection in CSA) and MoE feed-forward work; the lower mHC path constrains residual-lane mixing around both sublayers.

12.2. V4-Pro, V4-Flash, and Comparison Surfaces

V4-Pro and V4-Flash have different structural budgets. They differ in depth, hidden width, expert-pool size, selector width, and total and active parameter counts. Those differences change the amount and kind of computation available to a token; workload conclusions still require the same prompt set, inference policy, runtime configuration, and scorer.

The release gives V4-Pro 1.6T total parameters and 49B active parameters. It gives V4-Flash 284B total and 13B active [19]. Table 12.3 adds configuration fields that make the scale difference concrete. Both open-weight repositories declare `DeepseekV4ForCausalLM`. Pro gives the residual stack, expert pool, and CSA selector larger budgets [75, 73].

Compare Pro and Flash under the same workload, inference policy, runtime configuration, and scorer.

Artifact support and runtime support remain distinct. A runtime must implement the V4 model class, message encoding, CSA/HCA attention, MoE dispatch, and low-precision kernels before the configuration in the table becomes an executable serving path. Hardware capacity, kernel coverage, and structured output behavior then depend on the chosen implementation.

Definition 12.2.1 (Active parameter ratio) *For a sparse or conditionally active model with total parameter count P_{total} and per-token active parameter count P_{active}, the active parameter ratio is*

$$\rho = \frac{P_{\text{active}}}{P_{\text{total}}}.$$

It measures parameter activation, not latency or quality.

For example, applying this ratio to the reported counts gives $49/1600 \approx 3.1\%$ for Pro and $13/284 \approx 4.6\%$ for Flash [19]. Pro's smaller ratio does not make it cheaper or faster: it still activates 49B parameters per token rather than 13B, with a wider stack and larger selector budget. The ratios describe conditional activation, not a product ranking.

Concept-to-code map

The Pro and Flash specifications are grounded in the pinned configuration at `https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/config.json` and the corresponding Flash model-card record [75, 73].

- **Shape.** Width, depth, and expert fields make Flash a distinct configuration rather than a truncated Pro checkpoint.
- **Conditional work.** Expert counts and selector fields map stored capacity to the per-token active path.
- **Precision.** Quantization fields specify payload and scale interpretation that compatible kernels must preserve.
- **Runtime obligation.** Configuration fields identify required semantics; a serving engine must still implement the corresponding model and kernel paths.

Structural field	V4-Flash	V4-Pro
Total and active parameters	284B total; 13B active	1.6T total; 49B active
Decoder stack	43 layers; hidden size 4096	61 layers; hidden size 7168
Context field	`max_position_embeddings=1048576`	`max_position_embeddings=1048576`
CSA selector top-k	`index_topk=512`	`index_topk=1024`
MoE experts	256 routed, 1 shared, 6 active per token	384 routed, 1 shared, 6 active per token
mHC fields	`hc_mult=4`, `hc_sinkhorn_iters=20`	`hc_mult=4`, `hc_sinkhorn_iters=20`
Precision fields	`expert_dtype=fp4`; FP8 quantization config	`expert_dtype=fp4`; FP8 quantization config

Table 12.3.: Structural comparison of the V4-Flash and V4-Pro open-weight configurations [75, 73].

12.3. CSA, HCA, and Token-Wise Compression

The V4 release note compresses a large architecture story into the phrase "token-wise compression plus DSA." The V4 report expands that phrase into a hybrid attention design: Compressed Sparse Attention (CSA) and Heavily Compressed Attention (HCA) are interleaved across attention layers, while DeepSeek Sparse Attention (DSA) appears inside CSA as the sparse selector over compressed KV entries [19, 76]. The central questions are what is compressed, where selection is sparse or dense, and how retrieval changes after compression.

V4's report-backed long-context mechanism is hybrid: CSA compresses and sparsifies; HCA compresses more heavily and keeps dense attention over compressed entries.

First-pass picture. Think of V4 attention as three resolutions distributed across an interleaved stack. At any one hybrid layer, the scheduled core path is either CSA or HCA; Sliding-Window Attention (SWA) accompanies that core path and preserves exact recent state. CSA creates overlapping, medium-grained summaries and uses a learned selector to choose which summaries matter to the current query. HCA creates much coarser summaries but attends to all of them. Across the stack, the hybrid design combines exact local detail, selective global detail, and coarse global coverage without restoring dense attention over every original token.

The mechanisms change different objects. Token-wise compression replaces a span of token states with learned compressed entries. HCA changes which compressed entries are visible by rule. CSA adds learned selection over compressed entries. DSA names the inherited sparse-selection mechanism specialized inside the V4 design, while sliding-window attention preserves an exact local path. Compression changes representation; visibility and selection change which represented entries participate. This object map governs the detailed paths below.

Concept-to-code map

The hybrid-attention correspondence follows the official report at `https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf` and the pinned Pro configuration at `https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/config.json`.

- **Compression topology.** The report defines the CSA and HCA pooling spans, the layer schedule, and the parallel exact-window branch.

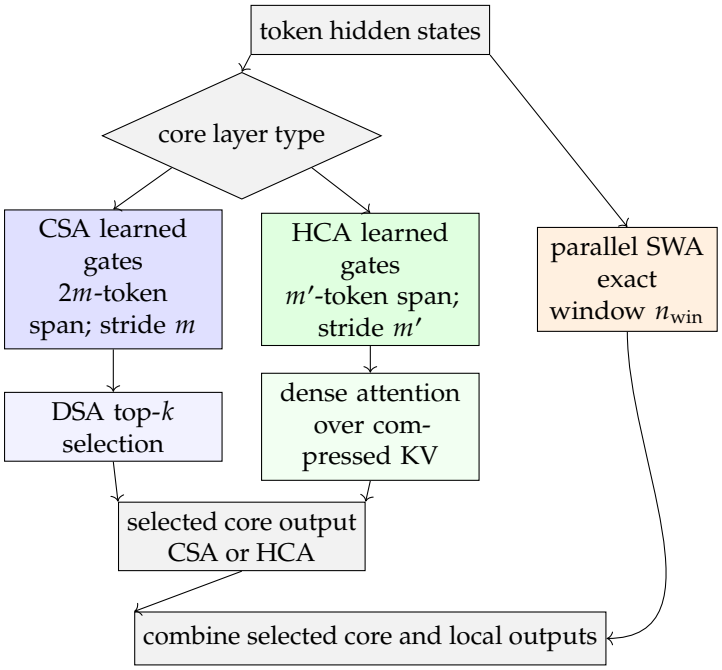


Figure 12.2.: CSA and HCA are alternative core paths assigned to different layers. A CSA layer advances by m positions while pooling two adjacent blocks and then applies sparse selection; an HCA layer pools non-overlapping m'-token groups and keeps dense attention over the shorter compressed sequence. Either core path combines with an uncompressed SWA local branch.

- **Selection budget.** The configuration's `index_topk` field records the artifact's selected-entry budget; the report defines which compressed objects it selects.
- **Context capacity.** The `max_position_embeddings` field records declared capacity, while the attention equations determine how eligible state is represented and read.
- **Implementation boundary.** Configuration fields identify required semantics but do not expose a production cache layout or kernel schedule.

Figure 12.2 first fixes the topology: one layer uses CSA or HCA as its core path, not both, and SWA runs in parallel with that choice. It then raises three first-pass questions: what each path compresses, whether it selects or reads every compressed entry, and why exact recent state remains available. The definitions and compressor equations below make those answers precise.

Definition 12.3.1 (Token-wise compression) *In this chapter, token-wise compression is the release-note name for V4's sequence-dimension compression of attention KV information. The report instantiates that idea through CSA and HCA: both use learned, coordinate-wise gates to pool source-token KV information before the attention step consumes the resulting entries [76].*

Definition 12.3.2 (Compressed Sparse Attention) *Compressed Sparse Attention, or CSA, emits one compressed KV entry per m source-token positions. Each entry pools two adjacent m-token blocks through two learned projection-and-gating streams, so its receptive span is $2m$ tokens while the compressed sequence has length about n/m. A DSA-style top-k selector then chooses which compressed entries a query token attends to. The V4 report also specifies compressed indexer keys, a lightning indexer, shared-KV multi-query attention, grouped output projection, and a sliding-window branch for local dependencies [76].*

Definition 12.3.3 (Heavily Compressed Attention) *Heavily Compressed Attention, or HCA, also compresses KV information along the sequence dimension, but pools non-overlapping groups at a much larger compression rate m'. Unlike CSA, HCA does not apply sparse attention over a selected subset; it performs dense attention over the heavily compressed entries, again with shared-KV MQA, grouped output projection, and a sliding-window branch [76].*

Definition 12.3.4 (Sliding-Window Attention) *Sliding-Window Attention, or SWA, preserves exact, uncompressed KV entries for the most recent n_{win} positions and limits local causal attention to that window. In a V4 hybrid layer, SWA runs alongside whichever compressed core path—CSA or HCA—the layer uses; V4-Flash also begins with two pure SWA layers. SWA keeps nearby dependencies available while compression groups are incomplete, but it cannot retrieve evidence that has fallen outside the local window [76].*

Definitions 12.3.1, 12.3.2, 12.3.3, and 12.3.4 fix the representation and selection roles before DSA is specialized to the V4 path.

DSA specialized to V4. Recall the V3.2 mechanism from Definition 11.3.1: a lightning indexer ranks prior positions and top-k selection admits their latent KV entries to the main attention path. V4 applies that selection rule inside CSA after KV compression. Its indexer ranks compressed KV entries that summarize source spans, rather than the per-position latent entries selected in V3.2, so selector recall and compression fidelity become separate tests [17, 19, 76].

The Learned Token-Level Compressor The compression rate is not the same thing as the number of source-token contributions to an entry. Using zero-based indices, let

$$A_i = \{mi, \dots, m(i+1)-1\}, \qquad P_i = \{m(i-1), \dots, mi-1\}$$

be the current and preceding m-token blocks for compressed index i. CSA projects the hidden states into two KV streams C^a, C^b and two data-dependent gate-logit streams Z^a, Z^b. With learned positional biases B^a, B^b, the report's compressor can be summarized as

$$\begin{bmatrix} S^a_{A_i} \\ S^b_{P_i} \end{bmatrix} = \text{softmax}_{\text{token}}\left(\begin{bmatrix} Z^a_{A_i} + B^a \\ Z^b_{P_i} + B^b \end{bmatrix}\right), \qquad C^{\text{comp}}_i = \sum_{j \in A_i} S^a_j \odot C^a_j + \sum_{j \in P_i} S^b_j \odot C^b_j.$$

The softmax normalizes, for each KV coordinate, across the $2m$ token contributions. A source block contributes through the a-stream to one compressed entry and through the b-stream to the next; that reuse creates the overlap while the output advances by m positions. Thus CSA with $m = 4$ has an eight-token receptive span, a four-token stride, and fourfold sequence compression. At the beginning of a sequence the unavailable preceding block is padded. HCA instead uses one gated stream over non-overlapping groups of $m' = 128$ tokens [76].

The local branch also closes a causal gap. At a query position, the current compression group may be incomplete; pooling it with positions that have not

Algorithm 18: One V4 hybrid compressed-attention state transition.

Input: Layer kind $\lambda \in \{\text{CSA}, \text{HCA}\}$; token state h_t; compressed, indexer, SWA, and incomplete-tail state; m, m', k, n_{win}
Output: Updated heterogeneous attention state and the layer attention output

```
project h_t into the layer's exact KV state and append it to the SWA and incomplete-tail records;
if λ = CSA and the next m-token group is complete then
    finalize the next overlapping 2m-token compressed entry with the two coordinate-wise gate streams;
    append its compressed KV and indexer-key records;
    release source state no longer needed by the next overlap, SWA, or tail;
else
    if λ = HCA and the next m'-token group is complete then
        finalize one non-overlapping HCA entry and append its compressed KV;
        release that completed group from the incomplete-tail record;
retain the declared recent exact window in the SWA state;
form the current latent query and the core-attention query representation;
if λ = CSA then
    score only finalized compressed entries with the lightning indexer;
    select top-k, gather those compressed entries, and execute sparse core attention;
else
    execute dense core attention over all finalized HCA entries;
execute local attention over the exact SWA state;
combine the core and local branches through the grouped output projection;
return the attention output and all updated cache classes;
```

arrived would leak future information. SWA therefore keeps exact recent KV state available until a group can be finalized [76].

The stored sequence is only one efficiency axis. Both paths first down-project the query hidden state to a latent c_t^Q and then up-project the core queries,

$$c_t^Q = h_t W^{DQ}, \qquad q_t = c_t^Q W^{UQ}.$$

CSA reuses c_t^Q for its indexer queries. Each compressed core-attention entry serves as both key and value, eliminating a separately cached value vector. The report applies partial rotary position encoding (RoPE) to queries and shared KV entries for scoring, then applies the corresponding inverse rotation to the core-attention output so absolute-position phase does not remain in the value content. After attention, both paths split the head outputs into groups, project each group through a smaller intermediate dimension, concatenate those intermediates, and apply the final output projection. Shared KV reduces stored state, while the query and output paths reduce projection work; none of these operations is the DSA selection itself [76].

Algorithm 18 begins by owning the uncompressed tail: newly arrived tokens enter exact local state before they are eligible for a compressed entry. A completed group then opens one of two gates. CSA finalizes an overlapping summary and an indexer key before top-k selection controls sparse core work; HCA finalizes a non-overlapping summary and reads every finalized summary. The SWA branch executes independently on recent exact state, and only then do the branches combine. The causal invariant is that an incomplete group never enters compressed attention; releasing source state is safe only after the next overlap, local window, and tail no longer own it.

The "gather" in Algorithm 18 is an indexed gather of the entries chosen by CSA's top-k selector. It is not the distributed all-gather used when long-context training partitions a sequence across context-parallel ranks. The dense-to-sparse curriculum and collective path are traced in Chapter 13.

12.4. From Residual Connections to mHC

A response schema describes what crosses the boundary of a run. Manifold-Constrained Hyper-Connections change residual-stream mixing inside a deep Transformer stack. This distinction matters because mHC is easy to misfile as another long-context attention trick. CSA, HCA, and DSA make attention-side choices such as selecting key/value positions and compressing KV blocks. mHC changes how layer inputs and residual paths are mixed so that depth and expressivity do not come only from unconstrained residual additions.

Before four lanes, picture two. A conventional residual block carries one row x and writes $x + F(x)$. A two-lane Hyper-Connection carries (x_1, x_2); a learned read map forms the input to F, and a learned write map distributes F's result back across both lanes. Extra lanes increase state-carrying capacity, but unconstrained reads and writes can amplify or erase the persistent path. mHC retains the multi-lane picture while constraining its mixing maps. The equations below formalize this two-lane intuition for the released four-lane design.

Start from the ordinary residual update around one Transformer sublayer $\mathcal{F}_l$:

$$x_{l+1} = x_l + \mathcal{F}_l(x_l). \tag{12.1}$$

The direct x_l term is an identity route. Expanding the recurrence from a shallower layer l to a deeper layer L gives

$$x_L = x_l + \sum_{i=l}^{L-1} \mathcal{F}_i(x_i).$$

The sublayers still transform the state, but the carried state is not itself multiplied by a learned residual map at every layer. The backward Jacobian likewise retains an identity contribution. This stable but rigid single-lane route is the baseline that HC tries to enrich.

The useful prehistory is Hyper-Connections, or HC. HC expands a Transformer's residual stream into several lanes and learns residual mappings between those lanes. That extra width gives the model more pathways for composing features, but it also creates a new stability problem: products of unconstrained residual mapping matrices can amplify or attenuate signals as depth grows [143]. mHC keeps the expanded residual lanes, but constrains the residual mixer so repeated layer-to-layer propagation remains bounded.

Write the n-lane residual state at layer l as m_l. One HC update can then be written

$$m_{l+1} = \mathscr{H}_l^{\text{res}} m_l + \left(\mathscr{H}_l^{\text{post}}\right)^{\top} \mathcal{F}_l\left(\mathscr{H}_l^{\text{pre}} m_l\right). \tag{12.2}$$

The row vector $\mathcal{H}_l^{\text{pre}}$ aggregates the lanes into the sublayer input, $\mathcal{H}_l^{\text{post}}$ chooses how the sublayer output is written back, and the $n \times n$ matrix $\mathcal{H}_l^{\text{res}}$ mixes the state carried around the sublayer. Only the last map is repeatedly multiplied along the direct residual route:

$$\mathcal{M}_{l \to L} = \mathcal{H}_{L-1}^{\text{res}} \mathcal{H}_{L-2}^{\text{res}} \cdots \mathcal{H}_l^{\text{res}}. \tag{12.3}$$

The danger is therefore a product, not one alarming-looking layer. As a scalar illustration rather than a paper measurement, a gain of only 1.05 repeated across 64 layers becomes $1.05^{64} \approx 22.7$.

Definition 12.4.1 (mHC) *V4 uses mHC to mix residual streams. The abbreviation means Manifold-Constrained Hyper-Connections. The V4 report describes the mechanism as expanding the residual stream by a small factor and constraining the residual mapping matrix to the manifold of doubly stochastic matrices. The official mHC paper frames the same choice as a correction to unconstrained HC: preserve the extra residual-lane mixing capacity while bounding signal propagation across depth [76, 143].*

$\mathcal{H}_l^{\text{res}}$ mixes residual lanes; $\mathcal{H}_l^{\text{pre}}$ and $\mathcal{H}_l^{\text{post}}$ connect the sublayer path.

Definition 12.4.2 (Doubly stochastic residual mixer) *A doubly stochastic residual mixer is a nonnegative matrix P whose rows and columns each sum to one:*

$$P_{i,j} \geq 0, \qquad \sum_j P_{i,j} = 1, \qquad \sum_i P_{i,j} = 1.$$

In the local mHC model, P mixes expanded residual lanes without creating an unconstrained amplification path.

The constraint is stronger than a cosmetic normalization. A doubly stochastic mixer is non-expansive in the sense used by the mHC paper, and the product of doubly stochastic matrices remains doubly stochastic. That closure property matters because a deep stack composes many residual mixers rather than a single isolated mixer. The Birkhoff-polytope intuition is also useful: the mixer can be read as a convex combination of lane permutations, so it can reroute residual information without creating an arbitrary gain path [143].

Figure 12.3 also fixes the mechanism boundary: the three $\mathcal{H}$ maps act around the Transformer sublayer; none selects which compressed KV entries attention scores.

The configuration fields make the abstraction tangible. The audited V4 Flash and Pro model cards expose `hc_mult=4` and `hc_sinkhorn_iters=20` [75, 73]. For intuition, `hc_mult=4` says to picture four residual lanes rather than one flat stream.

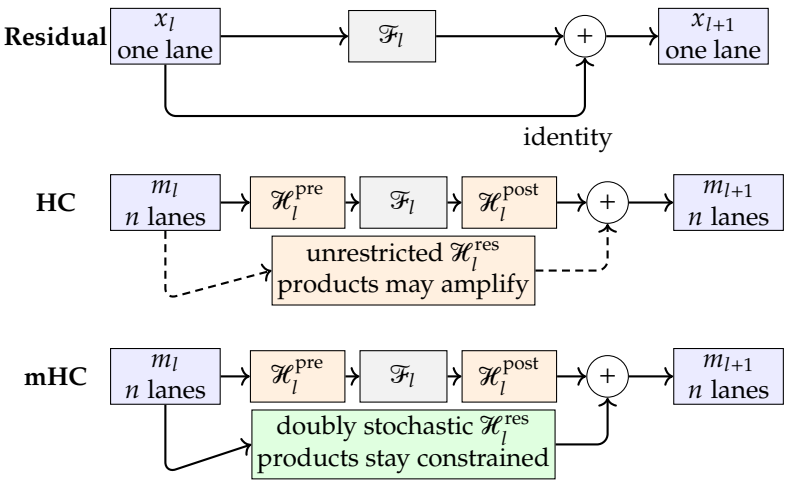


Figure 12.3.: Residual wiring as a progression. HC replaces the single identity lane with learned pre-, post-, and residual maps over multiple lanes. mHC keeps that routing capacity while constraining the residual-map products that compose through depth.

12.5. Sinkhorn-Constrained Residual Mixing

The three HC maps are not fixed matrices reused for every token. Using the book's row-vector convention $\mathcal{H}_l^{\text{post}} = C_l^\top$, V4 first flattens and normalizes the current residual state, $\widehat{m}_l = \text{RMSNorm}(\text{vec}(m_l))$, then combines an input-dependent projection with a learned static term:

$$\begin{aligned}
\widetilde{\mathcal{H}}_l^{\text{pre}} &= \alpha_l^{\text{pre}}(\widehat{m}_l W_l^{\text{pre}}) + S_l^{\text{pre}}, \\
\widetilde{\mathcal{H}}_l^{\text{res}} &= \alpha_l^{\text{res}}\,\text{Mat}(\widehat{m}_l W_l^{\text{res}}) + S_l^{\text{res}}, \\
\widetilde{\mathcal{H}}_l^{\text{post}} &= \alpha_l^{\text{post}}(\widehat{m}_l W_l^{\text{post}}) + S_l^{\text{post}}.
\end{aligned}$$

The constrained read and write maps are

$$\mathcal{H}_l^{\text{pre}} = \sigma(\widetilde{\mathcal{H}}_l^{\text{pre}}), \qquad \mathcal{H}_l^{\text{post}} = 2\sigma(\widetilde{\mathcal{H}}_l^{\text{post}}).$$

The raw residual map follows the Sinkhorn path below. This order matters: dynamic generation proposes the three maps, bounded nonlinearities constrain the read and write maps, and Sinkhorn normalization constrains the repeatedly composed residual carrier [76, 143].

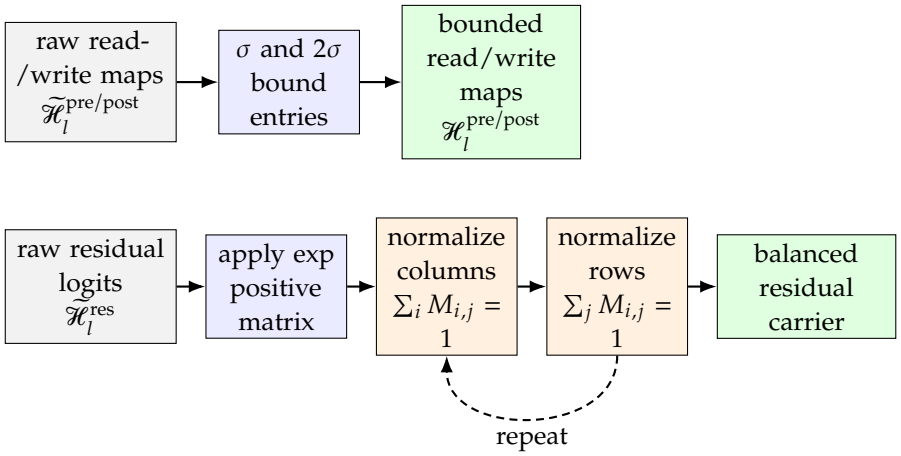


Figure 12.4.: The three generated maps enter two different constraint paths. Sigmoid bounds the read and write maps once; the residual map becomes positive and alternates column and row normalization until the finite iteration budget produces an approximately doubly stochastic carrier. The axis normalized last has the smaller terminal sum error.

Figure 12.4 separates the one-step bounded read/write path from the repeated balancing applied only to the residual map.

How Sinkhorn normalization reaches the constraint. Sinkhorn normalization is an alternating balancing procedure, not a one-shot division. Normalizing every column makes the column sums one, but the different column divisors generally disturb the row sums. Normalizing every row repairs the row sums while generally disturbing the columns. Repeating those two steps makes both sets of sums approach one.

For one explicit alternation, start from

$$M^{(0)} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}.$$

Column normalization gives

$$\mathcal{T}_c(M^{(0)}) = \begin{bmatrix} 2/3 & 1/2 \\ 1/3 & 1/2 \end{bmatrix},$$

whose column sums are exactly $(1, 1)$ and whose row sums are $(7/6, 5/6)$. Row normalization then gives

$$M^{(1)} = \begin{bmatrix} 4/7 & 3/7 \\ 2/5 & 3/5 \end{bmatrix}.$$

Its row sums are exactly $(1, 1)$, while its column sums are $(34/35, 36/35)$, closer to one than the original column sums $(3, 2)$. Another column normalization repairs the columns and again perturbs the rows; iteration reduces both residuals toward the finite implementation tolerance.

Let $L \in \mathbb{R}^{n \times n}$ be the raw learned mixing logits and start from the positive matrix $M^{(0)} = \exp(L)$. Define column and row normalization by

$$\left(\mathcal{T}_{\mathrm{c}}(M)\right)_{i,j} = \frac{M_{i,j}}{\sum_{i'} M_{i',j}}, \qquad \left(\mathcal{T}_{\mathrm{r}}(M)\right)_{i,j} = \frac{M_{i,j}}{\sum_{j'} M_{i,j'}}.$$

One Sinkhorn alternation is then

$$M^{(t)} = \mathcal{T}_{\mathrm{r}}\left(\mathcal{T}_{\mathrm{c}}\left(M^{(t-1)}\right)\right). \tag{12.4}$$

The mHC paper writes the column-then-row order shown here; an implementation can reverse the order, which only changes which axis is normalized last when the loop stops. Under the usual positivity conditions, continuing the alternation converges to a doubly stochastic matrix [143].

Real kernels stop after finitely many alternations. The `hc_sinkhorn_iters=20` field therefore describes an approximate projection, not a certificate that every row and column sum is symbolically one. Implementations can also add a small ϵ to entries or denominators to avoid unstable division by a tiny sum. That numerical floor improves robustness while introducing another small departure from exact unit sums. The architectural target remains the doubly stochastic family; the finite matrix is the numerical approximation used by the layer.

Finite Sinkhorn iterations approximate the doubly stochastic constraint; the last-normalized axis is exact only up to numerical error.

Example 12.5.1 (Two Sinkhorn alternations) Set $\epsilon = 0$ to keep the arithmetic exact and choose

$$L = \begin{bmatrix} 0 & \log 3 \\ 0 & 0 \end{bmatrix}, \qquad M^{(0)} = \exp(L) = \begin{bmatrix} 1 & 3 \\ 1 & 1 \end{bmatrix}.$$

The first column and row normalizations give

$$M^{(0)} \xrightarrow{\mathcal{T}_{\mathrm{c}}} \begin{bmatrix} 1/2 & 3/4 \\ 1/2 & 1/4 \end{bmatrix} \xrightarrow{\mathcal{T}_{\mathrm{r}}} M^{(1)} = \begin{bmatrix} 2/5 & 3/5 \\ 2/3 & 1/3 \end{bmatrix}.$$

The second alternation gives

$$M^{(1)} \xrightarrow{\mathcal{T}_{\mathrm{c}}} \begin{bmatrix} 3/8 & 9/14 \\ 5/8 & 5/14 \end{bmatrix} \xrightarrow{\mathcal{T}_{\mathrm{r}}} M^{(2)} = \begin{bmatrix} 7/19 & 12/19 \\ 7/11 & 4/11 \end{bmatrix}.$$

The final row sums are exactly one because row normalization was last. The column sums are 210/209 and 208/209, already close to one but not equal after only two alternations. This is the finite-iteration distinction hidden by the shorthand "project onto the doubly stochastic manifold."

Exact rational arithmetic can preserve every fraction in the two-alternation trace. The checkpoint applies the chapter's column-then-row order explicitly, rather than calling a matrix-balancing routine whose stopping convention is hidden.

Executable checkpoint: Two exact Sinkhorn alternations

```python
from fractions import Fraction as F

Matrix = tuple[tuple[F, ...], ...]


def normalize_columns(matrix: Matrix) -> Matrix:
    totals = tuple(sum(row[j] for row in matrix) for j in range(len(matrix[0])))
    return tuple(
        tuple(value / totals[j] for j, value in enumerate(row)) for row in matrix
    )


def normalize_rows(matrix: Matrix) -> Matrix:
    return tuple(tuple(value / sum(row) for value in row) for row in matrix)


def sinkhorn_step(matrix: Matrix) -> Matrix:
    return normalize_rows(normalize_columns(matrix))


m0 = ((F(1), F(3)), (F(1), F(1)))
after_columns_1 = normalize_columns(m0)
m1 = normalize_rows(after_columns_1)
after_columns_2 = normalize_columns(m1)
m2 = normalize_rows(after_columns_2)

assert after_columns_1 == ((F(1, 2), F(3, 4)), (F(1, 2), F(1, 4)))
assert m1 == ((F(2, 5), F(3, 5)), (F(2, 3), F(1, 3)))
assert after_columns_2 == ((F(3, 8), F(9, 14)), (F(5, 8), F(5, 14)))
assert m2 == ((F(7, 19), F(12, 19)), (F(7, 11), F(4, 11)))
assert tuple(map(sum, m2)) == (F(1), F(1))
column_sums = tuple(sum(row[j] for row in m2) for j in range(2))
assert column_sums == (F(210, 209), F(208, 209))

if __name__ == "__main__":
    matrix_text = "; ".join("(" + ", ".join(map(str, row)) + ")" for row in m2)
    column_text = ", ".join(map(str, column_sums))
    print(f"two-step matrix={matrix_text}; column sums=({column_text})")
```

Run: `python3 chapter_deepseek_v4/v4_sinkhorn_checkpoint.py`

The assertions recover both column-normalized intermediates, $M^{(1)}$, and $M^{(2)}$. They also verify unit final row sums and the residual column sums 210/209 and 208/209. The executable trace and its separate test are in `chapter_deepseek_v4/`. This 2-by-2, zero-epsilon fixture establishes finite-iteration arithmetic, not a V4 kernel, numerical stability result, or training reproduction.

The current residual state first generates raw input, residual, and output maps in Algorithm 19; this is the input-dependent preparation step, not yet the Transformer

Algorithm 19: One conceptual mHC sublayer transition. Dynamic map generation is followed by bounded read/write maps, finite Sinkhorn balancing of the residual carrier, sublayer execution, and the residual-plus-write commit.

Input: Residual state m_l; sublayer $\mathcal{F}_l$; learned dynamic-map projections, static terms, and gates; Sinkhorn count $t_{\max}$
Output: Next residual state m_{l+1} and the constrained maps used by the transition

flatten and RMS-normalize m_l to obtain $\widehat{m}_l$;
generate $\widetilde{\mathcal{H}}_l^{\text{pre}}$, $\widetilde{\mathcal{H}}_l^{\text{res}}$, and $\widetilde{\mathcal{H}}_l^{\text{post}}$ from their dynamic and static terms;
set $\mathcal{H}_l^{\text{pre}} \leftarrow \sigma(\widetilde{\mathcal{H}}_l^{\text{pre}})$ and $\mathcal{H}_l^{\text{post}} \leftarrow 2\sigma(\widetilde{\mathcal{H}}_l^{\text{post}})$;
set $M \leftarrow \exp(\widetilde{\mathcal{H}}_l^{\text{res}})$;
for *iteration* $s = 1$ **to** $t_{\max}$ **do**
    normalize every column of M to unit sum;
    normalize every row of M to unit sum;
set $\mathcal{H}_l^{\text{res}} \leftarrow M$;
read the lanes and execute the sublayer, $u_l \leftarrow \mathcal{F}_l(\mathcal{H}_l^{\text{pre}} m_l)$;
carry the residual lanes, write the sublayer result, and set $m_{l+1} \leftarrow \mathcal{H}_l^{\text{res}} m_l + (\mathcal{H}_l^{\text{post}})^\top u_l$;
return m_{l+1} *and the three constrained maps*;

sublayer work. Sigmoid then bounds the read and write maps while finite Sinkhorn alternation makes the residual carrier approximately doubly stochastic. Then, the read map forms the model-width input and $\mathcal{F}_l$ performs the attention or feed-forward work. The constrained carrier and write term commit one new n-lane state. The safety invariant belongs to the carrier: products of the ideal residual maps remain doubly stochastic, while a finite implementation must test its row- and column-sum error. The complete state may still change because the sublayer write term injects new content.

12.6. Composing mHC Across Transformer Sublayers

One constrained carrier is not enough to establish stability through depth. The remaining question is how successive attention and feed-forward sublayers compose their residual maps while new writes continue to change the state. The following trace makes that multi-sublayer boundary explicit.

For one token, an attention sublayer first reads a mixture of the four incoming lanes. CSA or HCA computes the attention update from its selected state. The attention write map distributes that update back into the lane carrier. The feed-forward sublayer then reads the resulting lanes, preserves routed-token identity through MoE dispatch and combine, and writes its update through a second constrained map. Only after both writes does the next block receive the new four-lane state. Correct isolated operators are insufficient if their lane boundaries disagree. The numerical audit below instantiates this order.

Example 12.6.1 (A four-lane, two-sublayer mHC conservation audit) Suppress the token index and display one hidden coordinate of a four-lane state. Write one mHC transition as

$$r_{l+1} = P_l r_l + p_l u_l, \qquad u_l = \mathcal{F}_l(a_l^\top r_l), \qquad r_l \in \mathbb{R}^4.$$

The vector a_l reads the four lanes into the sublayer, p_l writes its scalar output at the displayed coordinate back into the lanes, and P_l carries and mixes the residual state. Let

$$C = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \qquad P_0 = 0.6I + 0.4C, \qquad P_1 = 0.5I + 0.3C + 0.2C^2.$$

The cyclic shift C and its powers are permutation matrices. Each P_l is therefore a convex combination of permutation matrices and is doubly stochastic.
Start from $r_0 = (4, 0, -2, 2)^\top$. For an attention sublayer, choose

$$a_0 = (0.5, 0.25, 0.125, 0.125)^\top, \qquad a_0^\top r_0 = 2, \qquad u_0 = \mathscr{F}_0(2) = 1,$$

and $p_0 = (0.8, 0.4, 0.6, 0.2)^\top$. The first update is

$$r_1 = P_0 r_0 + p_0 u_0 = (4, 2, -0.6, 0.6)^\top.$$

For the following feed-forward sublayer, choose

$$a_1 = (0.1, 0.2, 0.3, 0.4)^\top, \qquad a_1^\top r_1 = 0.86, \qquad u_1 = \mathscr{F}_1(0.86) = -0.5,$$

and $p_1 = (0.2, 0.6, 0.4, 0.8)^\top$. Then

$$r_2 = P_1 r_1 + p_1 u_1 = (1.96, 2.02, 0.90, 0.12)^\top.$$

The useful audit is the two-layer path decomposition

$$r_2 = \underbrace{P_1 P_0 r_0}_{(1.48,\,1.84,\,0.52,\,0.16)^\top} + \underbrace{P_1 p_0 u_0}_{(0.58,\,0.48,\,0.58,\,0.36)^\top} + \underbrace{p_1 u_1}_{(-0.10,\,-0.30,\,-0.20,\,-0.40)^\top}.$$

Only the first term is the direct residual route. Its composite map is

$$M = P_1 P_0 = 0.30I + 0.38C + 0.24C^2 + 0.08C^3,$$

so $M\mathbf{1} = \mathbf{1}$, $\mathbf{1}^\top M = \mathbf{1}^\top$, and

$$\|M\|_1 = \|M\|_\infty = \|M\|_2 = 1, \qquad \operatorname{spec}(M) = \{1,\ 0.08,\ 0.06 \pm 0.30\mathrm{i}\}.$$

The constant lane mode is preserved. Every mean-zero mode in this constructed circulant example contracts by at most $\sqrt{0.06^2 + 0.30^2} \approx 0.306$. The complete state sum changes from 4 to 6 and then to 5 because the two $p_l u_l$ terms inject and remove sublayer output. That change does not violate the constraint on the carried P_l path.
For contrast, set the didactic unconstrained-HC control to $Q_l = 1.05P_l$. Then

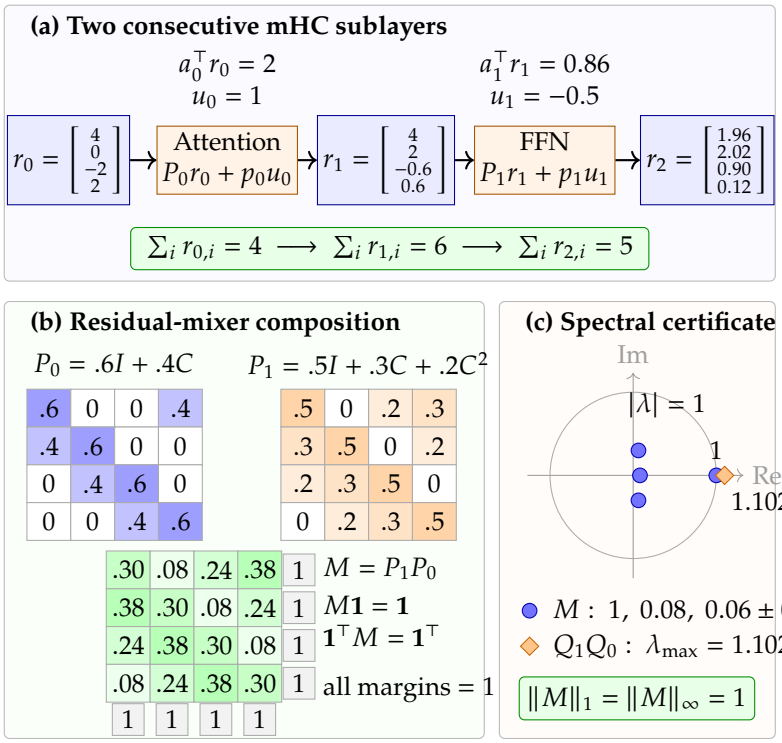
$$\|Q_1 Q_0\|_1 = \|Q_1 Q_0\|_\infty = 1.1025.$$

Its leading mode has already moved outside the non-expansive unit disk after two sublayers. These coefficients are an exact architectural trace, not recovered

Residual design	Added capacity	Control point	Diagnostic implication
Ordinary residual stream	One main path around each block	One residual lane per block	Baseline propagation
Unconstrained HC	Multiple lanes with learned lane mixing	Composed residual maps can amplify or attenuate signal across depth	Composite gain can grow rapidly
mHC	HC-style lanes plus a doubly stochastic mixer	Repeated composition stays inside the constrained mixer family	Composite gain remains bounded

Table 12.4.: mHC keeps the extra residual-lane capacity of Hyper-Connections while constraining the mixer that is repeatedly composed through depth.

V4 weights; finite Sinkhorn normalization in a real layer only approximates the doubly stochastic target.



In the two-layer decomposition, $P_1 P_0$ is carried residual mixing; $P_1 p_0 u_0$ and $p_1 u_1$ are injected sublayer outputs.

Figure 12.5.: A four-lane mHC state transition across two Transformer sublayers. The read and write maps add transformed content, while the composed direct residual map remains doubly stochastic. Its unit eigenvalue preserves the lane mean and its other modes are contractive; the scaled HC control moves the leading mode outside the non-expansive unit disk.

Figure 12.5 makes the ownership of the conservation claim explicit. The row and column margins belong to the composed residual map M, not to the complete r_2 update after the two write terms.

Table 12.4 distinguishes the three residual paths. Ordinary residual paths, unconstrained HC, and constrained mHC form a progression that the official paper tests numerically. In its 120-layer, expansion-rate-four diagnostic, the reported composite residual-map growth under unconstrained HC can approach 3000, while the mHC version stays around 1.6 [143]. Those are training diagnostics, not a guarantee about V4 latency, retrieval, or downstream quality.

Warning 12.6.1 (mHC is residual mixing, not retrieval)**.** mHC constrains residual mixing and signal propagation. It does not imply that V4 will retrieve a specific

clause from a one-million-token prompt, reduce a CSA selector miss, or improve a workload score.

mHC is also not free. Equation 12.2 still feeds one model-width state into the expensive attention or feed-forward sublayer, so it does not multiply the core sublayer width by n. The carried residual state, however, grows from one model-width lane to n lanes. The pre-, post-, and residual maps must read and write that widened state; activation checkpoints must save or recompute it; and pipeline boundaries may have to communicate it. The practical pressure is therefore memory traffic and interstage communication as well as arithmetic. The paper treats fused kernels, activation recomputation, and pipeline overlap as part of the mechanism story. Its V4-scale report says the optimized path adds 6.7% marginal training overhead for mHC [143]. That is a training-systems quantity, not a serving-cost or run-latency number.

Speculative decoding must also honor the residual-state invariant. The next section treats that serving mechanism separately. mHC therefore remains a residual-stability concept rather than a catch-all label for V4 inference.

12.7. Architecture Interactions and Open Questions

The individual operators meet at one token boundary. CSA or HCA produces a long-range attention result, SWA contributes exact local state, mHC determines which residual lanes the attention and feed-forward sublayers read and write, and DeepSeekMoE preserves routed-token identity inside the inherited sparse feed-forward path. A shape-correct implementation can still be wrong when two neighboring operators disagree about positions, selected entries, or lanes.

Across that block path, compressed positions, selector indices, residual lanes, and routed buffers must describe the same token state. Attention selection must be causal; the gathered entries must match the selected indices and positions; mHC transitions must preserve the lane contract; MoE dispatch and combine must preserve routed-token identity; and the final block must return the declared model width. These are architecture invariants. Optimizer moments, QAT masters,

Table 12.5.: V4 checkpoint-architecture boundaries that require joint tests.

Interaction	Shared state or boundary	Diagnostic failure
CSA/HCA + DSA	Compressed entries, selector scores and indices, exact SWA state, positions	Legal tensor shapes with the wrong compressed span, selection, or positional layout
mHC + depth	Four residual lanes and constrained mixing matrices	Residual gain grows, collapses, or loses lane semantics
Attention + mHC	Selected attention output, lane reads, lane writes, and block output	Correct isolated attention composed with the wrong lane boundary
mHC + MoE	Feed-forward input lanes, routed-token identity, combined expert output, residual write	Correct routes or expert outputs written to the wrong carrier state

specialist teachers, tool schemas, and cache schedulers belong to later training, agent, or runtime surfaces.

Scope 12.7.1 (Engram remains adjacent research)**.** The released V4-Pro and V4-Flash mechanism set is backed by the report and artifacts: mHC, CSA, HCA, DSA, DeepSeekMoE, and MTP. Those sources mention Engram only as a future direction, not a released component [9, 76, 75, 73].

The architecture-side open work is empirical: selector recall, compression fidelity, causal gather correctness, the interaction between exact local and compressed long-range paths, and mHC stability through depth. Matched training ablations and precision studies continue in Chapter 13; serving latency, cache state, and speculative accepted length continue in the V4 runtime chapter.

An integrated failure can remain shape-correct: the selector returns legal compressed indices, gather reads the corresponding cache rows, and attention produces one model-width update, but the implementation writes that update through the feed-forward lane map instead of the attention lane map. Every local tensor shape and selected position can look valid while depth-wise residual semantics drift. A joint reference test must therefore retain token positions, selected ids, gathered rows, lane reads, lane writes, routed-token ids, and final block output in one trace.

12.8. Summary

V4's checkpoint architecture combines hybrid CSA/HCA attention, a DSA selector over compressed entries, exact sliding-window state, mHC residual mixing, and inherited DeepSeekMoE and MTP components. Pro and Flash use the same mechanism classes at different depths, widths, expert counts, and selector budgets.

One token therefore carries several identities at once: original and compressed positions, selected-entry indices, exact local KV state, four residual lanes, and routed-expert records. Table 12.5 is the joint-test map for keeping those identities aligned. It does not import Muon, QAT masters, specialist teachers, agent adapters, or speculative draft state into the architecture.

The next V4 chapter explains how the dense-to-sparse curriculum, context parallelism, Muon/AdamW ownership, QAT, specialist rewards, and on-policy distillation construct the released policies. Engram remains adjacent research rather than a released V4 component.

V4's architecture has four distinct changes to retain. CSA and HCA operate on compressed long-range state, with an exact sliding-window path preserving local context. DSA supplies learned fine-grained selection inside the compressed attention design. mHC replaces a single residual stream with four constrained lanes across attention and feed-forward sublayers. Pro and Flash instantiate different width, depth, expert, and selector budgets while sharing the reported mechanism family. None of these structural facts alone ranks their quality or serving speed.

12.9. Exercises

1. Core: Classify hybrid CSA/HCA attention, mHC, DeepSeekMoE, MTP, Muon, MXFP4 QAT, specialist distillation, and speculative decoding as a V4 architecture change, inherited component, training procedure, or optional serving path. Explain why the categories are not interchangeable.
2. Core: Compare V4-Pro and V4-Flash using total and active parameters, routed-expert count, active routed experts per token, `index_topk`, `hc_mult`, and `hc_sinkhorn_iters`. Separate structural facts from latency, resource, retrieval, and quality claims that still require measurement.
3. Applied: Explain CSA, HCA, DSA, and SWA. For a CSA layer with compression rate (m=4), identify the raw span represented by one compressed entry, the object ranked by DSA, the exact state retained by SWA, and the causal-selection and gather invariants.
4. Applied: A query at original position 20 may select compressed entries with source spans (1:8), (9:16), (17:24), and (21:28). Determine which spans are legal under a causal gather rule, state what positional record must accompany each retained entry, and explain why an in-range top-(k) index is not by itself sufficient evidence of correctness.
5. Applied: Let $P = \begin{bmatrix} 0.6 & 0.4 \\ 0.4 & 0.6 \end{bmatrix}$ and $Q = \begin{bmatrix} 1.3 & 0.2 \\ 0.1 & 1.1 \end{bmatrix}$. Check whether each is doubly stochastic and compute P^3v, Q^3v for $v = (1, 0)^\top$. Then run the Sinkhorn checkpoint, apply a third step to $M^{(2)}$, and report exact row sums and column residuals.
6. Challenge: Trace one token through an attention sublayer and a feed-forward sublayer with four mHC residual lanes. Name the pre-map, dynamic input, post-map, residual write, and block-output boundary. Give one isolated-operator test that would pass while the composed block is wrong.
7. Challenge: Construct a joint test for CSA/HCA, mHC, and DeepSeekMoE that records compressed positions, selected indices, residual lanes, routes, combine weights, and final width. Name two faults that separate operator tests would miss.
8. Challenge: Build an evidence table for CSA/HCA, mHC, DeepSeek-MoE/MTP, Engram, Muon, and speculative decoding. For each row, record whether it is released architecture, inherited architecture, adjacent research, training, or runtime, plus one falsifying or still-open test.